

Data Structures And Algorithms

Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 03- Jan- 2025

LAB NO 12

Task # 1

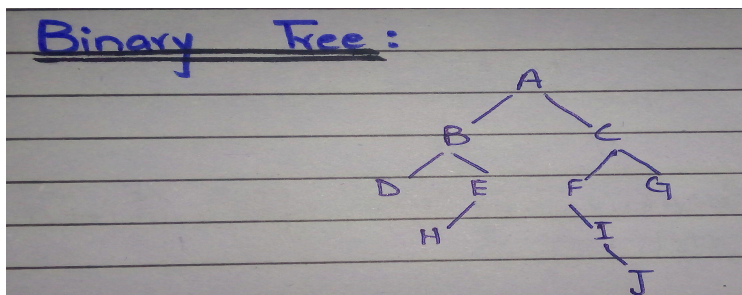
Give answers to the following.

Question 1:

Given the following two traversals, attempt to construct the binary tree. Hints: In preorder traversal, the first node is the root (while in postorder traversal the last node is the root node). Once root is identified, all nodes in the left and right sub-trees of root can be identified (repeatedly).

Inorder: D B H E A I F J C G

Preorder: A B D E H C F I J G



Question 2:

State what the following function is intended to compute.

```
int A_Recursive_Counter(TNode *root)
{
    if(root==NULL)
        return 0;
```

```
else  
return 1 + A_Recursive_Counter(root->left) + A_Recursive_Counter(root->right);  
}
```

The function `A_Recursive_Counter` is intended to compute the number of nodes in a binary tree

It works recursively:

- if the root is Null, it returns 0.
- otherwise, it returns 1, plus the sum of the counts of the nodes in the left and right subtrees.

Question 3:

What is the drawback if a sorted list of integers is used to generate a BST ?

- Inserting a sorted line into a BST creates a skewed tree that resembles a linked list.
- The tree's height becomes n , making search, insert and delete operation run in $O(n)$ time.
- This defeats the purpose of BST, which should provide $O(\log n)$ performance for balanced tree.
- The resulting structure wastes space and reduces overall efficiency of operations.
- To prevent this, use a balancing technique like AVL or red-black insertion.

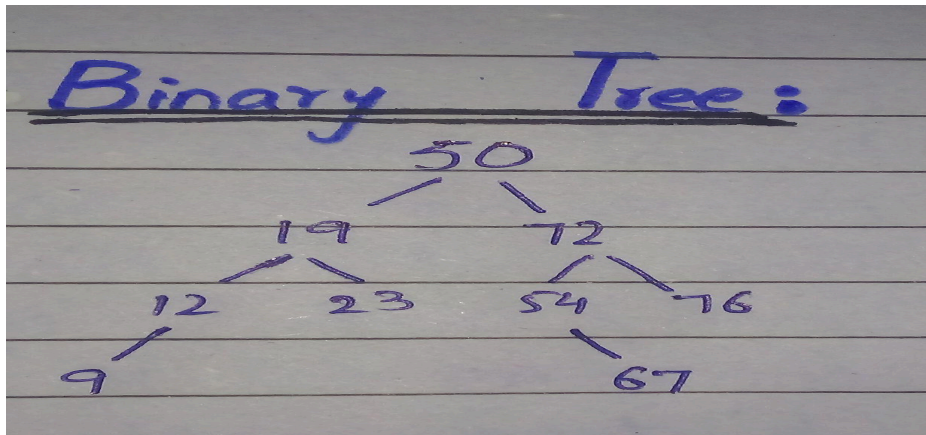
Question 4:

A binary tree has a height of 5. What is the minimum number of nodes it can have?

The minimum number of nodes in a binary tree of height h is $h+1$
for a tree of height 5:
$$= 5 + 1$$
$$= 6$$

Question 5:

Re-draw the given BST after deleting the node containing the value 17.

**Task # 2**

Implement the following exercises.

Exercise 1

Write program that creates a binary search tree using the BST class developed in the previous having the function `void DeleteNode(Node* temp, int num)` verify the output by deleting every possibility of node.

PROGRAM:

```
#include <iostream>
using namespace std;
```

```
struct Node {
    Node *left, *right;
    int data;
    Node() { left = right = NULL; }
    Node(int item) {
        data = item;
        left = right = NULL;
    }
};

class bst {
    Node *root;
public:
    bst() { root = NULL; }
    bool isempty() {
        return (root == NULL);
    }
};
```

```

void insert(int item) {
    Node *newNode = new Node(item);
    if (isempty()) {
        root = newNode;
        return;
    }
    Node *ptr = root;
    Node *parent = NULL;
    while (ptr != NULL) {
        parent = ptr;
        if (item < ptr->data)
            ptr = ptr->left;
        else if (item > ptr->data)
            ptr = ptr->right;
        else {
            // Item already exists, do not insert
            delete newNode;
            return;
        }
    }
    if (item < parent->data)
        parent->left = newNode;
    else
        parent->right = newNode;
}

bool search(int item) {
    Node *ptr = root;
    while (ptr != NULL) {
        if (item < ptr->data)
            ptr = ptr->left;
        else if (item > ptr->data)
            ptr = ptr->right;
        else
            return true; // Item found
    }
    return false; // Item not found
}

void Preorder(Node *ptr) {
    if (ptr != NULL) {
        cout << ptr->data << " ";
        Preorder(ptr->left);
        Preorder(ptr->right);
    }
}

```

```

void Postorder(Node *ptr) {
    if (ptr != NULL) {
        Postorder(ptr->left);
        Postorder(ptr->right);
        cout << ptr->data << " ";
    }
}

void Inorder(Node *ptr) {
    if (ptr != NULL) {
        Inorder(ptr->left);
        cout << ptr->data << " ";
        Inorder(ptr->right);
    }
}

Node* findMin(Node* node) {
    while (node->left != NULL)
        node = node->left;
    return node;
}

Node* DeleteNode(Node* temp, int num) {
    if (temp == NULL)
        return temp;
    if (num < temp->data)
        temp->left = DeleteNode(temp->left, num);
    else if (num > temp->data)
        temp->right = DeleteNode(temp->right, num);
    else {
        // Case 1: No child
        if (temp->left == NULL && temp->right == NULL) {
            delete temp;
            temp = NULL;
        }
        // Case 2: One child
        else if (temp->left == NULL) {
            Node* temp2 = temp;
            temp = temp->right;
            delete temp2;
        }
        else if (temp->right == NULL) {
            Node* temp2 = temp;
            temp = temp->left;
            delete temp2;
        }
        // Case 3: Two children
    }
}

```

```

        else {
            Node* temp2 = findMin(temp->right);
            temp->data = temp2->data;
            temp->right = DeleteNode(temp->right, temp2->data);
        }
    }
    return temp;
}

void Delete(int num) {
    root = DeleteNode(root, num);
}

// Public interface for traversals
void PreorderTraversal() { Preorder(root); }
void PostorderTraversal() { Postorder(root); }
void InorderTraversal() { Inorder(root); }
};

int main() {
    bst tree;
    tree.insert(50);
    tree.insert(30);
    tree.insert(20);
    tree.insert(40);
    tree.insert(70);
    tree.insert(60);
    tree.insert(80);

    cout << "Inorder traversal: ";
    tree.InorderTraversal();
    cout << endl;

    // Delete node with no child
    tree.Delete(20);
    cout << "Inorder traversal after deleting 20: ";
    tree.InorderTraversal();
    cout << endl;

    // Delete node with one child
    tree.Delete(30);
    cout << "Inorder traversal after deleting 30: ";
    tree.InorderTraversal();
    cout << endl;

    // Delete node with two children

```

```

tree.Delete(70);
cout << "Inorder traversal after deleting 70: ";
tree.InorderTraversal();
cout << endl;

// Delete root node
tree.Delete(50);
cout << "Inorder traversal after deleting 50: ";
tree.InorderTraversal();
cout << endl;
return 0;
}

```

OUTPUT:

```

Inorder traversal: 20 30 40 50 60 70 80
Inorder traversal after deleting 20: 30 40 50 60 70 80
Inorder traversal after deleting 30: 40 50 60 70 80
Inorder traversal after deleting 70: 40 50 60 80
Inorder traversal after deleting 50: 40 60 80

```

Exercise 2

Write a program that creates a binary search tree using the BST class developed in the previous lab session. Write functions to implement the following.

1. Find the minimum value in the BST
2. Find the maximum value in the BST
3. Given an integer value, find the address of its parent node.

PROGRAM:

```

#include <iostream>
using namespace std;

struct Node {
    Node *left, *right;
    int data;
    Node() { left = right = NULL; }
    Node(int item) {
        data = item;
        left = right = NULL;
    }
}

```

```

};
class bst {
    Node *root;
public:
    bst() { root = NULL; }

    bool isempty() {
        return (root == NULL);
    }
    void insert(int item) {
        Node *newNode = new Node(item);
        if (isempty()) {
            root = newNode;
            return;
        }
        Node *ptr = root;
        Node *parent = NULL;
        while (ptr != NULL) {
            parent = ptr;
            if (item < ptr->data)
                ptr = ptr->left;
            else if (item > ptr->data)
                ptr = ptr->right;
            else {
                // Item already exists, do not insert
                delete newNode;
                return;
            }
        }
        if (item < parent->data)
            parent->left = newNode;
        else
            parent->right = newNode;
    }
    bool search(int item) {
        Node *ptr = root;
        while (ptr != NULL) {
            if (item < ptr->data)
                ptr = ptr->left;
            else if (item > ptr->data)
                ptr = ptr->right;
            else
                return true; // Item found
        }
    }

```



```

        return false; // Item not found
    }
    void Preorder(Node *ptr) {
        if (ptr != NULL) {
            cout << ptr->data << " ";
            Preorder(ptr->left);
            Preorder(ptr->right);
        }
    }
    void Postorder(Node *ptr) {
        if (ptr != NULL) {
            Postorder(ptr->left);
            Postorder(ptr->right);
            cout << ptr->data << " ";
        }
    }
    void Inorder(Node *ptr) {
        if (ptr != NULL) {
            Inorder(ptr->left);
            cout << ptr->data << " ";
            Inorder(ptr->right);
        }
    }
    int findMin() {
        if (isempty()) {
            cout << "Tree is empty" << endl;
            return -1;
        }
        Node *ptr = root;
        while (ptr->left != NULL)
            ptr = ptr->left;
        return ptr->data;
    }
    int findMax() {
        if (isempty()) {
            cout << "Tree is empty" << endl;
            return -1;
        }
        Node *ptr = root;
        while (ptr->right != NULL)
            ptr = ptr->right;
        return ptr->data;
    }
    Node* findParent(int item) {

```

```

    if (isempty() || root->data == item) {
        cout << "Parent not found" << endl;
        return NULL;
    }
    Node *ptr = root;
    Node *parent = NULL;
    while (ptr != NULL) {
        if (item < ptr->data) {
            parent = ptr;
            ptr = ptr->left;
        }
        else if (item > ptr->data) {
            parent = ptr;
            ptr = ptr->right;
        }
        else
            return parent; // Item found, return parent
    }
    cout << "Item not found" << endl;
    return NULL;
}

// Public interface for traversals
void PreorderTraversal() { Preorder(root); }
void PostorderTraversal() { Postorder(root); }
void InorderTraversal() { Inorder(root); }
};

int main() {
    bst tree;
    tree.insert(50);
    tree.insert(30);
    tree.insert(20);
    tree.insert(40);
    tree.insert(70);
    tree.insert(60);
    tree.insert(80);

    cout << "Inorder traversal: ";
    tree.InorderTraversal();
    cout << endl;

    cout << "Minimum value: " << tree.findMin() << endl;
    cout << "Maximum value: " << tree.findMax() << endl;

    Node *parent = tree.findParent(40);

```

```

if (parent != NULL)
    cout << "Parent of 40: " << parent->data << endl;

parent = tree.findParent(50);
if (parent != NULL)
    cout << "Parent of 50: " << parent->data << endl;
else
    cout << "50 is the root node" << endl;

return 0;
}

```

OUTPUT:

```

Inorder traversal: 20 30 40 50 60 70 80
Minimum value: 20
Maximum value: 80
Parent of 40: 30
50 is the root node

```

Exercise 3

An effective way to represent Mathematical expressions is using the binary trees. The following algorithm can be employed to generate an expression tree from a given postfix expression.

1. Scan the postfix expression from left to right.
2. Create a node Curr
3. Get a symbol E from the expression.
4. If the symbol is an operand Set this operand as data member of the node Curr Push the node on the stack
5. If the symbol is an operator T2 = Pop()

PROGRAM:

```

#include <iostream>
#include <stack>
#include <string>
using namespace std;
class TNode {
public:
    char data;
    TNode *left;
    TNode *right;
    TNode(char c) {
        data = c;
        left = right = NULL;
    }
}

```

```

    }
};
TNode* constructTree(string postfix) {
    stack<TNode*> s;
    for (char c : postfix) {
        TNode *curr = new TNode(c);
        if (isOperand(c)) {
            s.push(curr);
        }
        else {
            TNode *t2 = s.top();
            s.pop();
            TNode *t1 = s.top();
            s.pop();
            curr->left = t1;
            curr->right = t2;
            s.push(curr);
        } }
    return s.top();
}
bool isOperand(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
}
void postorder(TNode *ptr) {
    if (ptr != NULL) {
        postorder(ptr->left);
        postorder(ptr->right);
        cout << ptr->data;
    }
}

int main() {
    string postfix = "ab+ef*g*-";
    TNode *root = constructTree(postfix);
    cout << "Postorder traversal: ";
    postorder(root);
    cout << endl;
    return 0;
}

```

OUTPUT:

```
Postorder traversal: ab+ef*g*-
```